# Student Council
# **Election Tally System**

A Java console application that automates voting and result tabulation.

**Team No :**

**Greeshma Kolagotla : 2620040163**
**G.L.B. Ananya : 2620040168**

# Project
# **Abstract**

The Student Council Election Tally System is a Java console-based application that automates voting and result tabulation. It manages candidate information, accepts votes, and computes results to determine the winner, demonstrating core Java concepts ,such as data types, arrays, control flow classes, sorting and streams through a .practical real-world application

Next Slide

AUTOMATE. COUNT. DECLARE.

# Problem & **Solution**

***FROM MANUAL COUNTING TO AUTOMATED RESULTS.***

Elections need a fast, accurate and auditable way to count votes.

## Problem

,Student council elections often rely on manual vote counting which is time-consuming and prone to human error. Counted by hand, results are also difficult to verify, which can lead to doubts about fairness. A faster, more accurate and more .reliable way to record and count votes is needed

## Solution

This project is a Java console application that automates the voting and counting process. It displays the candidates, accepts one vote per voter and checks that each choice is valid, asking again if it isn’t. Votes are counted automatically, and the program then shows each candidate’s total, announces the winner and lists .the candidates in sorted order

# CO1
# Topics

*FOUNDATIONS OF JAVA*

Primitive data types (int, String)
Variables and 1D arrays
Operators
Console I/O (Scanner, println)

```java
public class ElectionTallySystem2 {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        String[] candidates = {"Candidate A", "Candidate B", "Candidate C"};
        int[] votes = new int[3];

        System.out.println(x: "Student Council Election Tally System");
        System.out.print(s: "Enter total number of voters: ");
        int totalVoters = sc.nextInt();
```

Declares primitive types and 1D arrays for candidates and votes, reads the voter count with Scanner, and prints prompts .using console output

# CO2
# Topics

*CONTROL FLOW*

for loops (voting iteration)
if-else (input validation)
Nested loops
(&&) Logical operator

```java
for (int i = 0; i < totalVoters; i++) {
    System.out.println("\nVoter " + (i + 1) + " - Select your choice:");
    for (int j = 0; j < candidates.length; j++) {
        System.out.println((j + 1) + ". " + candidates[j]);
    }
    System.out.print(s: "Enter choice (1-3): ");
    int choice = sc.nextInt();

    if (choice >= 1 && choice <= 3) {
        votes[choice - 1]++;
    } else {
        System.out.println(x: "Invalid choice!");
        i--;
    }
}
```

,A for loop handles each voter, a nested loop prints the menu and if-else with && checks the range. Decrementing i re-asks .the same voter on invalid input

CO3

# Topics

*ARRAYS AND ALGORITHMS*

1D array traversal
Searching and comparing values
Finding the maximum value
Comparison operators

```
    System.out.println(x: "_____ELECTION RESULTS_____");
    int maxVotes = votes[0];
    int winner = 0;

    for (int i = 0; i < votes.length; i++) {
        System.out.println(candidates[i] + ": " + votes[i] + " votes");
        if (votes[i] > maxVotes) {
            maxVotes = votes[i];
            winner = i;
        }
    }

    System.out.println("\nWinner: " + candidates[winner] + " with " + maxVotes + " votes");

    sc.close();
}
```

Shows array traversal using for loop, searching/comparing values, finding maximum using conditional logic, and tracking .the winner index for result display

C04

# Topics

*OBJECT ORIENTED BASICS*

Classes and objects
Fields and parameterized constructor
this keyword
Object creation with new
Object references stored in a List

```
class Candidate {
    String name;

    Candidate(String name) {
        this.name = name;
    }
}
```

```
for (int i = 0; i < votes.length; i++) {
    Candidate c = new Candidate(candidates[i]);
    results.add(c);
    System.out.println(c.name + ": " + votes[i] + " votes");
```

The Candidate class is a blueprint with one field. Its constructor uses this, and each object is created with new and stored in a .list

# CO5
# Topics

*STRINGS AND LIBRARY ALGORITHMS*

String arrays and concatenation
Arrays.sort() library sorting
()Copying arrays with clone
Enhanced for loop over Strings

```
String[] sorted = candidates.clone();
Arrays.sort(sorted);
System.out.println(x: "\nCandidates in sorted order:");
for (String name : sorted)
    System.out.println(name);
```

Sorts a copy of the names with Arrays.sort(), so the original array stays aligned with votes, then prints them with an .enhanced for loop

C06

# Topics

*COLLECTIONS AND STREAMS*

Collections: List, ArrayList
Generics (List<Candidate>)
Lambda expressions
()Streams: filter(), forEach

```
        System.out.println(x: "\nCandidates who received votes:");
        results.stream()
            .filter(c -> votes[Arrays.asList(candidates).indexOf(c.name)] > 0)
            .forEach(c -> System.out.println(c.name));

        sc.close();
    }
}
```

Candidates are kept in a type-safe ArrayList. A stream pipeline filters those with at least one vote using a lambda and prints .each name

# Project
# Conclusion

The Election Tally System shows how core Java concepts (variables, loops, arrays, classes, sorting and streams) combine to solve a real problem. It validates each vote, tallies results automatically and announces the winner, replacing slow manual counting. Future scope: any number of candidates, tie handling, .file storage and a graphical interface

# Screenshots



```
greeshmak@Greeshmas-MacBook-Air Java Practicals %  cd /Users/greeshmak/Desktop/Java\ Practicals ; /usr/bin/env /Library/Java/JavaVirtualMachines/jdk-26.jdk/Contents/Home/bin/java -agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:55866 --enable-preview -XX:+ShowCodeDetailsInExceptionMessages -cp /Users/greeshmak/Library/Application\ Support/Code/User/workspaceStorage/0ea68458bb054600ac453777a2b5e824/redhat.java/jdt_ws/Java\ Practicals_962c5d1b/bin ElectionTallySystem2
Student Council Election Tally System
Enter total number of voters: 3

Voter 1 - Select your choice:
1. Candidate A
2. Candidate B
3. Candidate C
Enter choice (1-3): 1

Voter 2 - Select your choice:
1. Candidate A
2. Candidate B
3. Candidate C
Enter choice (1-3): 1

Voter 3 - Select your choice:
1. Candidate A
2. Candidate B
3. Candidate C
Enter choice (1-3): 3

ELECTION RESULTS
Candidate A: 2 votes
Candidate B: 0 votes
Candidate C: 1 votes

Winner: Candidate A with 2 votes

Candidates in sorted order:
Candidate A
Candidate B
Candidate C

Candidates who received votes:
Candidate A
Candidate C
```

# Git-Hub LINK

https://github.com/Greeshma-x9/Java-Assignment

## Geo-tag Photo

# Thank you